Procedural Noise and Textures

Computer Graphics Course Project: Summary and Plan (Team 11, CG_04)

KESHAV SINGH, IIIT Delhi, India
KARANVEER SINGH, IIIT Delhi, India

We will implement gradient noise (with value and simplex noise as alternative bases) and use it to build fBm, turbulence, marble, wood, cloud and terrain on the GPU, add noise-based bump mapping, and study how noise type, octaves and persistence affect quality and cost.

Authors' Contact Information: Keshav Singh, keshav24300@iiitd.ac.in, IIIT Delhi, India; Karanveer Singh, karanveer24294@iiitd.ac.in, IIIT Delhi, India.

Index

Abstract 1
Index 2
1 Project Topic and Reference Paper 3
2 Problem and Objective 3
3 Primary Implementation 3
4 Proposed Extension 3
5 Proposed Experiment 3
6 Implementation Plan 4
7 Expected Challenges 4
8 Team Responsibilities 4
References 4

1 Project Topic and Reference Paper

Procedural Noise and Textures (CG_04). Reference: K. Perlin, "An Image Synthesizer," *Proc. ACM SIGGRAPH '85*, Computer Graphics 19(3), pp. 287–296, DOI 10.1145/325334.325247 [1]. We also use Perlin's "Improving Noise" [2] for the revised interpolant and Inigo Quilez's articles [3] for practical fBm, domain-warping and terrain techniques.

2 Problem and Objective

Image-based textures need storage, tile visibly and are hard to vary. The paper instead synthesizes surface detail procedurally: a band-limited pseudo-random *gradient noise* $n(\mathbf{x})$ is evaluated per sample and scaled copies are summed to mimic the self-similar structure of natural materials, giving resolution-independent, parameterizable textures at negligible memory cost. We will implement this pipeline in a real-time shader framework, produce a small family of materials, and study quantitatively how the noise parameters control appearance and cost.

3 Primary Implementation

- **Gradient noise** in 2D and 3D (lattice gradients, dot products with offsets, smooth interpolation), with both the original cubic $3t^2 - 2t^3$ and the quintic $6t^5 - 15t^4 + 10t^3$ fade.
- **Value noise** as a baseline behind the same interface, so any base noise can feed the fractal sums.
- **Fractal sums:** fBm $\sum_i p^i n(2^i \mathbf{x})$ with adjustable octaves and persistence p, and turbulence $\sum_i p^i |n(2^i \mathbf{x})|$.
- **Textures:** marble (sine plus turbulence), wood (perturbed rings) and clouds (fBm with a colour ramp) on 3D objects using object-space coordinates.
- **Basic terrain:** an fBm height field on a displaced grid mesh, finite-difference normals, height/slope colouring (water, grass, rock, snow) and directional lighting.

4 Proposed Extension

Main: noise-driven bump mapping with animation. We perturb the surface normal using the gradient of the fBm height field (analytic or finite-difference) before Phong lighting, giving apparent relief without extra geometry; time as a third noise axis animates clouds and surfaces. **Alternatives** (chosen by progress): (a) *domain warping*, $f(\mathbf{p} + \mathbf{w}(\mathbf{p}))$ with $\mathbf{w}$ an fBm, for flowing marble, smoke and terrain; (b) 2D/3D simplex noise as a further base; (c) erosion-like derivative-based terrain.

5 Proposed Experiment

Question: how do octave count N, persistence p and the base noise type (value, Perlin, simplex) affect visual quality and cost, and beyond which octave does extra detail stop being perceptible?

Varied: $N \in \{1, \ldots, 12\}$, $p \in \{0.3, 0.5, 0.7\}$, base frequency, fade curve, and base noise type. **Measured:** (i) GPU frame time (OpenGL timer queries, fixed resolution, averaged over many frames); (ii) RMSE of N octaves against a 12-octave reference, plus side-by-side renderings; (iii) the radially and angularly averaged power spectrum of rendered fBm images (a small C++ DFT tool on exported images).

Expected: cost grows roughly linearly with N; spectral falloff follows a power law set by p; the image converges after a few octaves, and octaves above the pixel Nyquist limit add aliasing rather than detail. Value noise should look blocky and grid-aligned, Perlin noise smoother but still somewhat axis-aligned, and simplex noise the most isotropic and cheaper per sample in higher dimensions. Cubic fading should show faint grid-aligned artifacts under bump mapping that quintic removes.

6 Implementation Plan

Stack: C++, OpenGL 3.3 core, GLSL, GLFW, GLAD, GLM. **Input:** no external data; simple meshes (plane, sphere, cube, terrain grid) with positions, UVs and normals; all textures are computed in shaders. **Stages:** (1) scene framework with camera and uniform-driven parameters; (2) hash-based gradient and value noise behind a common interface, validated by visualizing raw noise; (3) fBm, turbulence, materials and terrain, plus 2D simplex noise; (4) bump mapping and animation; (5) experiment harness (parameter sweeps, timer queries, image export, C++ analysis tools); (6) report and demo scenes. **Simplifications:** an integer hash replaces Perlin's permutation table; we run in real time on the GPU rather than in an offline shading language; octave fading by pixel footprint only if time permits.

7 Expected Challenges

(1) *Aliasing:* high octaves alias under minification, so a justified per-pixel octave cutoff (e.g. from screen-space derivatives) needs care. (2) *Hash quality and fair measurement:* weak GPU hashes cause repetition or directional artifacts, and timer-query results are noisy, so runs must be controlled and repeated.

8 Team Responsibilities

Keshav: noise core (hashing, value/Perlin/simplex, interpolants), fBm/turbulence, materials, terrain height field. **Karanveer:** rendering framework, terrain mesh and shading, bump-mapping/animation (and domain warping if chosen), experiment harness and analysis tools. Both will review each other's code and be able to explain every component.

References

[1] Ken Perlin. 1985. An Image Synthesizer. *Computer Graphics (SIGGRAPH '85)* 19, 3, 287–296. https://doi.org/10.1145/325334.325247
[2] Ken Perlin. 2002. Improving Noise. *ACM Trans. Graph. (SIGGRAPH '02)* 21, 3, 681–682.
[3] Inigo Quilez. Articles on procedural graphics. https://iquilezles.org/articles/